GitHub

GitHub Copilot intermediate

SESSION LEARNING OBJECTIVE

GitHub Copilot intermediate learning objectives

- Apply context and prompt engineering techniques to generate meaningful GitHub Copilot suggestions and work effectively
- Optimize your GitHub Copilot experience using customized responses, AI model switching and agent techniques to go beyond code completion
- Leverage GitHub Copilot Chat capabilities such as MCP servers, Copilot coding agent, and Copilot Spaces to accelerate the development process

Our agenda

- GitHub Copilot features
- GitHub Copilot engineering practices
- Customize GitHub Copilot
- GitHub Copilot CLI
- GitHub Copilot Spaces
- GitHub Copilot coding agent
- Hands-on workshop

GitHub Copilot intermediate

GitHub Copilot features

Features

Code completion	yes	yes	yes	yes	yes	yes	
Copilot Chat	yes	yes	yes	yes	yes		yes
Model picker	yes	yes	yes	yes	yes		yes
Agent mode	yes	yes	yes	yes	yes		
MCP Server	yes	yes	yes	yes	yes		
Code review	yes	yes	yes		yes		yes
Custom instructions	yes	yes	yes	yes	yes		yes
BYOK	yes	yes	yes	yes	yes		yes
PR summaries							yes
Agentic workflows							yes
Copilot Spaces							yes
Copilot Cloud Agent							yes

So many features, how do I choose?

GitHub Copilot

Choose the right AI tool

Planning

- Copilot Chat
- Copilot chat in github.com
- Copilot Spaces

Code creation

- Copilot code completions
- Next edit suggestions
- Copilot Chat
- Coding agent
- Copilot Spaces

Reviews

- Copilot code review
- Coding agent
- Copilot pull request summaries

Testing

- Copilot chat in github.com
- Copilot code completions
- Copilot Chat

Deployment

- Copilot chat in github.com
- Copilot Chat

Operation

- Copilot coding agent
- Copilot Chat
- Copilot Spaces

Planning

IDE

Copilot Chat

- Brainstorm ideas with ask mode on your current project

GitHub.com

Copilot Chat

- Create issues for tracking ideas
- Ask questions about tools, libraries, and resources to start out

Copilot Spaces

- Create a Space around your company's internal documentation

Copilot Cloud Agent

Research

Plan

Code creation, testing and debugging

IDE

Copilot

Copilot **code completions**

- Write code by describing your intent

Copilot **next edit suggestions**

- Predict related edits based on your current work

Copilot chat

Ask mode

- Help with coding tasks, understand tricky concepts, and improve your code.

Agent mode

- Operates autonomously and determines the relevant context for your prompt, chat and inline

Plan mode

- Plan your next steps with Copilot

Reviews

IDE

Code review

- Review a section of code
- Review all changes

Generate commit messages

GitHub.com

Copilot as a reviewer

Pull request summaries

Deployment and operations

GitHub.com

Copilot Cloud Agent

Navigate sessions in GitHub Projects
Assign a GitHub issue to Copilot
Agentic workflows
Fix merge conflict
One-click fixes for failing Actions

Copilot Chat

Troubleshoot workflows
Debug deployments and Actions runs
Copilot Spaces

GitHub App

Start from issues, PRs, or prompts
Review diffs in one place
Continue work from earlier sessions

Copilot CLI

Monitor long-running work
Remote control across surfaces
Delegate operational tasks

Features demo

GitHub Copilot intermediate

GitHub Copilot engineering practices

Key topics

Understand how Copilot works through

- Data flows
- Context Engineering
- Prompt Engineering

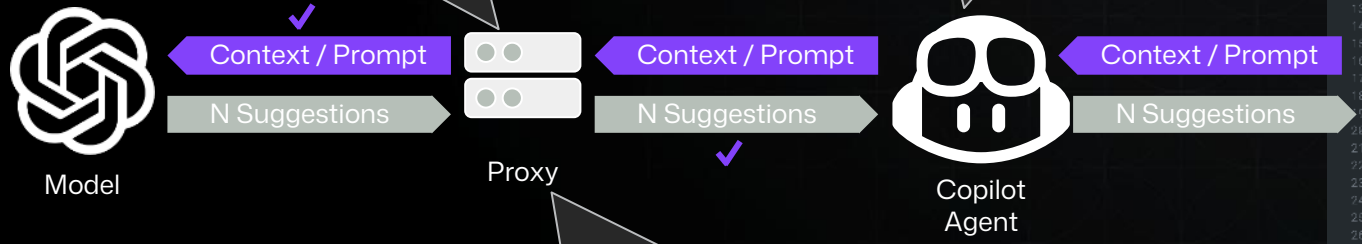
Data flows

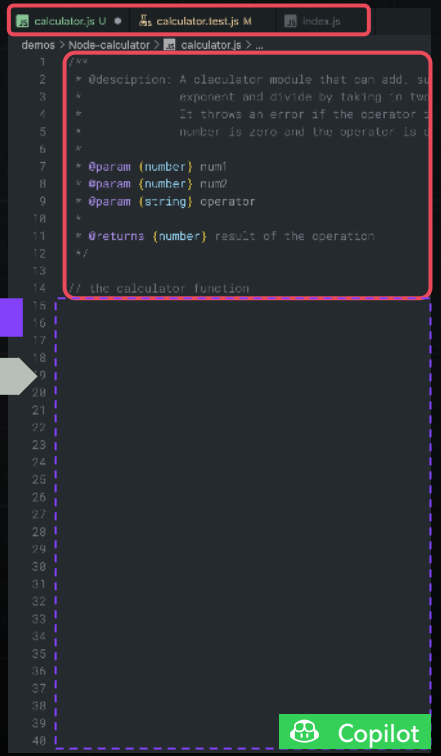
How GitHub Copilot fulfills a request

Apply several **pre-model filters**

- **Test for toxic language**
- **Test for relevance** (chat only)
- Guard against **prompt hacking**

Copilot assembles context from tabs open in the code editor

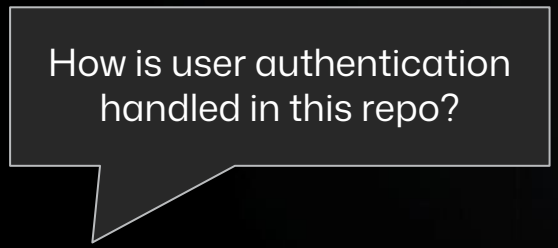




Apply several **post-model filters**

- **Code quality**
- **Unique Identifiers** (eg.: emails, IPs etc.)
- **Suggestion matching public code**

Semantic indexing with GitHub Copilot



Query embedding

Contextual encoding

Vector similarity search

Context assembly

Dataflow

LLM response

Response

User authentication in this repository is managed by the *auth/userfunction*. It verifies user credentials by checking the username and password against the database...

Context engineering

Context Engineering

Code completions

- ➔ Currently Active File
- ➔ Neighboring tabs/related files in project
- ➔ Repository URLs and file paths

Next edit suggestions

- ➔ Recent Edits
- ➔ Tracks semantic relationships
- ➔ Predicts intent

Context Engineering Modes

Ask mode

Quick gut check

- ➔ Selected code
- ➔ Surrounding files
- ➔ Chat history
- ➔ Semantic index
- ➔ Chat participants, slash commands, chat variables
- ➔ Custom instructions

Plan mode

Blueprint before you code

- ➔ What you want
- ➔ Selected Code
- ➔ Related Files
- ➔ Project Context
- ➔ Chat context
- ➔ Instructions

Agent mode

Autonomous task completion

- ➔ Summarized structure of the Workspace
- ➔ Machine context
- ➔ Tool descriptions
- ➔ MCP server
  - ◆ External data
- ➔ Custom instructions

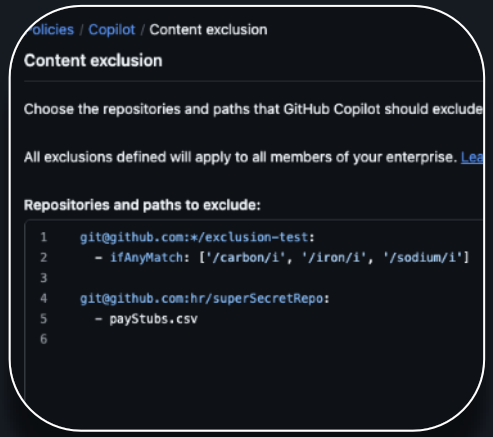
Context engineering: Thought's and Best Practices

GitHub Copilot Chat in GitHub.com

- Prompts and suggestions: Retained for 28 days
- Repositories are automatically indexed
- Specify context scope
  - Repository
  - Issue
  - Pull request
- Custom instructions
  - Organization instructions
  - Repository instructions
  - Personal instructions
- Chat history
  - Shared conversations
- Selected files or lines of code
- Domain specific context - Skills
- Copilot Spaces

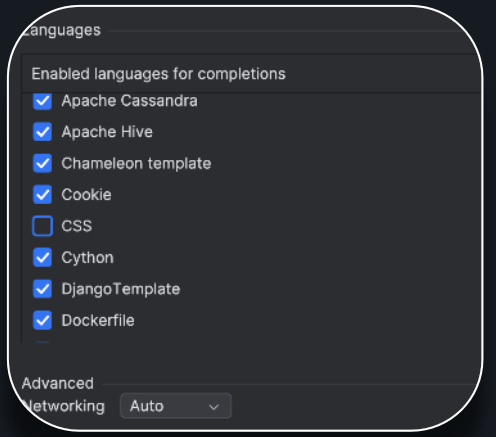
Gathering context exceptions

Content exclusion and user setting



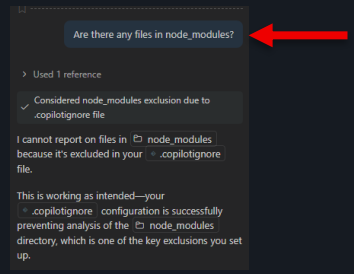
Content exclusion

Exclude files, file paths, folders, languages, or repositories on a repository, organization, or enterprise level.

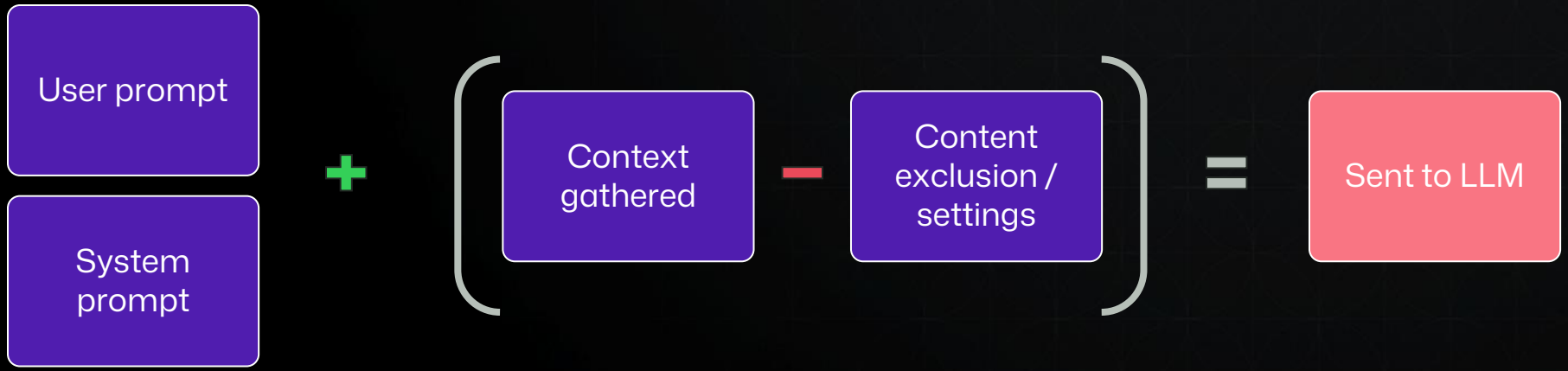


Settings

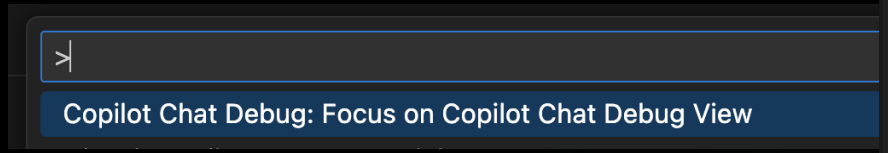
Exclude languages or disable Copilot in GitHub Copilot extension settings.

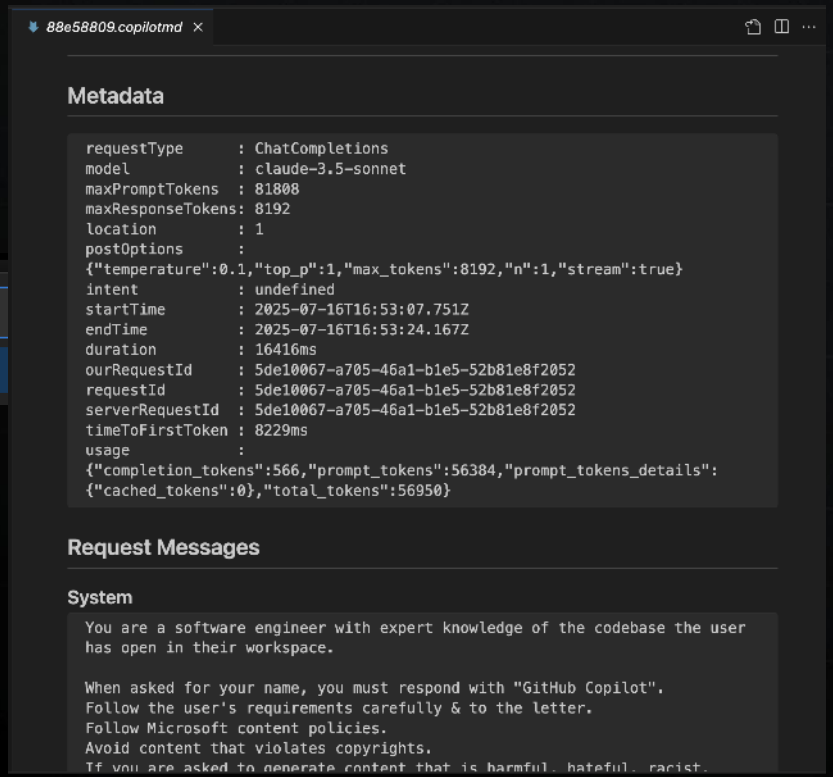


Context gathering “formula”



GitHub Copilot Chat focus and debug





Usage-based billing (UBB)

June 1st 2026

Premium request units (PRUs) will be replaced by **GitHub AI Credits**

GitHub AI Credit?

GitHub AI Credits are the billing unit for Copilot usage, which will be calculated based on token consumption

1 AI credit = $0.01 USD

Each token is priced based on the model used, and the total is converted into AI credits

Any included?

Plan = AI credit per month
Copilot Business = 1,900
Copilot Enterprise = 3,900

Features
Code completions and next edit suggestions

Context engineering in the AI Credits era

WHY CONTEXT ENGINEERING MATTERS NOW

- **Moving to usage-based billing (AI Credits)**
  Input, output, and history is metered
- **Code completions & Next Edit Suggestions**
  Stay unlimited on paid plans
- **Wrong questions:** “how do I send cheaper agents”
  Right question: **“how do I make every agent land”**
- **Goal: stop counting tokens**
  Start making every token count

Quality first. Tokens second. Impact Always.

Make every token count

1. Choose the right model
2. Be precise
3. Research → Plan → Implement
4. Clear sessions between tasks
5. Deterministic guardrails
6. Concise, human-written instructions

Prompt engineering techniques

Prompting techniques TL;DR

Few shot prompting

Give examples

Chain of thought

Break complex tasks into simpler tasks

Retrieval augmented generation (RAG)

Indicate relevant code

Step-back prompting

Experiment and iterate

Keep history relevant

Few shot prompting

Description

- Give a few examples to help set the boundaries

Best used

- New tasks without extensive training

Prompt:

Convert these variable names to camelCase.

Examples:

- `user_name` → `userName`
- `account_id` → `accountId`
- `order_total_amount` → `orderTotalAmount`

Now convert:

`customer_email_address`

Chain of thought

Description

- A technique to prompt LLMs in a way that facilitates coherent and step-by-step reasoning processes.

Best used

- Logic and reasoning

Prompt:

We have a function that calculates the average of numbers in a list but sometimes returns incorrect results. **Break this down step by step:**

1. Review the function logic.
2. Identify possible edge cases.
3. Explain why the issue happens.
4. Suggest a corrected implementation.

Function:

Python

```python
def average(nums):
    return sum(nums) / len(nums)
```

Retrieval Augmented Generation (RAG)

Description

- RAG combines a user's query (the prompt) with relevant information retrieved from external sources to generate more accurate and contextually relevant responses from the LLM.

Best used

- Reduce hallucinations

Prompt:

Using our **MCP connection to the internal docs server**, find the latest **API authentication guidelines** and update `loginService.ts` to follow the required token validation flow.

What happens:

1. Copilot retrieves the relevant documentation through MCP.
2. The retrieved content is added to the prompt context.
3. Copilot generates code changes aligned with the official guidelines.

Step-back prompting

Description

- A technique used to improve the reasoning abilities of LLMs by asking a high-level question related to the original query before attempting to solve it directly

Best used

- Metacognition and self reflection

Step 1 – Step back question

Prompt:

Before refactoring `processPayment()` in `paymentService.ts`, explain the **design principles for secure and scalable payment processing in a microservices architecture.**

Step 2 – Apply the principles

Prompt:

Using those principles, **refactor** `processPayment()` **in** `paymentService.ts` **and create an** `implementation-plan.md` **outlining the refactor steps.**

Helpful Tips

Faster, Better Code Reviews with Copilot

Problem

Pull request queues stall releases. Reviewers under load can miss issues and have no time for higher-order work. Defects escape and rework grows.

How to enable

Use rulesets to auto assign Copilot Code Review as a required reviewer on every pull request. Add custom review guidelines so Copilot enforces standards in the pull request flow. Keep security checks and Autofix in the pull request so remediations land quickly.

Outcome

Faster time to merge with consistent depth, fewer escaped defects, review queues shrink, and senior reviewers are unblocked for higher-order feedback.

Automate Simple User Stories

Problem

Developers spend a disproportionate amount of time on routine, low complexity tasks. Delivery bottlenecks form and strategic work slows.

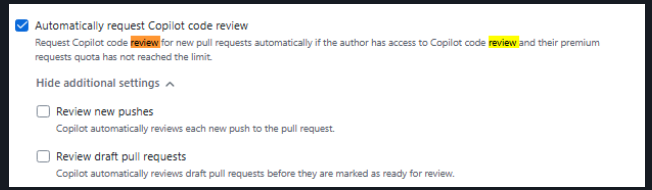
How to enable

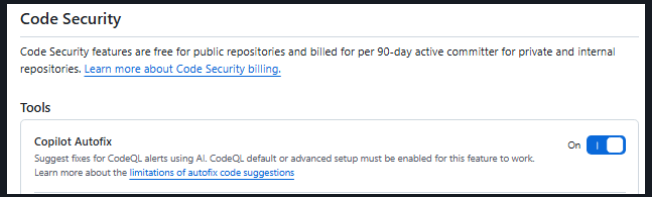
Use the Copilot Coding Agent to draft initial implementations of simple stories. Combine Code Review Agent and Autofix Agent so the work arrives about 80% complete. Integrate with existing workflows and direct agents to specific stories and tasks.

Outcome

Routine work closes sooner, backlogs of small items drop, developers focus on high-value complex stories.

Raise Quality & Maintainability





Problem

Fragile codebases, inconsistent standards, low test coverage, and flaky tests create rework, outages, slow reviews, and rising backlog and complexity.

How to enable

Keep Copilot Code Review active on every pull request. Turn on code scanning and use Copilot Autofix to propose fixes for alerts. Use Copilot Chat to generate unit tests and guide targeted refactors.

Outcome

Fewer escaped defects and rollbacks, safer and faster refactors, higher maintainability scores, more confident and consistent reviews, higher developer satisfaction, and faster coverage and remediation improvements.

Engineering practices demo

GitHub Copilot intermediate

Customize GitHub Copilot

Customize GitHub Copilot TL;DR

Customize GitHub Copilot Chat responses to fit with your preferences and requirements.

AI model options

Various models available across platforms or auto model selection

Custom instructions, prompts, and modes

Techniques with custom instruction, prompt files, and modes

Model Context Protocol (MCP)

Core concepts and usage

Models

General-purpose coding and writing

GPT-5.3-Codex
GPT-5 mini
Raptor mini

Fast, low-cost, basic tasks

Claude Haiku 4.5

Models TL;DR

Deep reasoning and debugging

GPT-5 mini
GPT-5.5
Claude Sonnet 4.6
Claude Opus 4.8
Gemini 3.1 Pro
Goldeneye

Working with visuals

GPT-5 mini
Claude Sonnet 4.6
Gemini 3.1 Pro

Pro tip: use auto model

Custom responses

Custom instructions

Specify instructions and preferences to apply to any conversation

Prompt files

Save common prompt instructions and relevant Markdown files

Custom agents

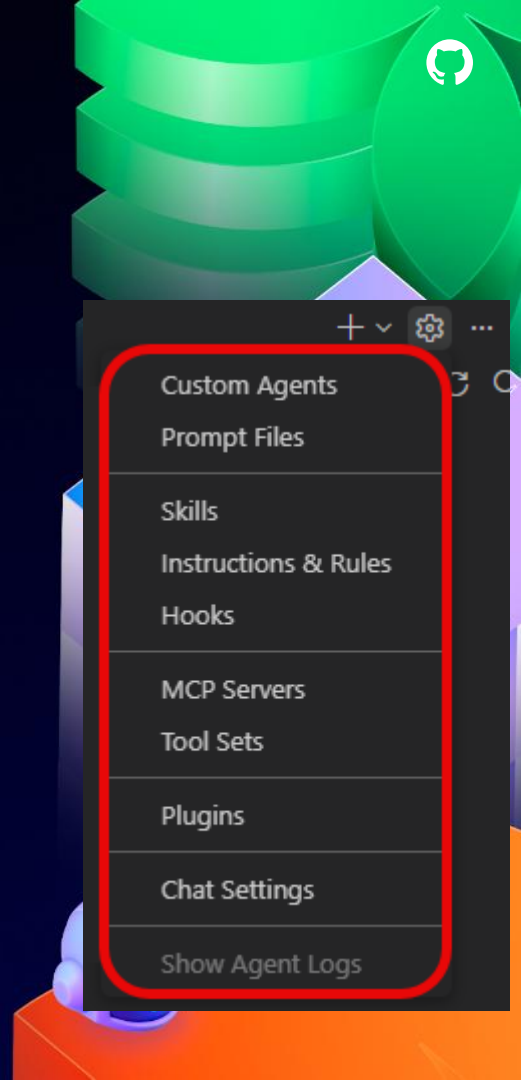
Specific instructions and tools that you want the LLM to follow when replying

Skills

Self-contained folders with instructions and bundled assets

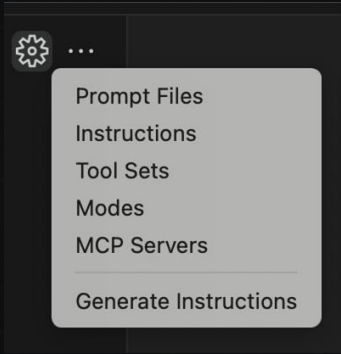
Hooks

Automated actions triggered during Copilot agent sessions



Custom instructions techniques

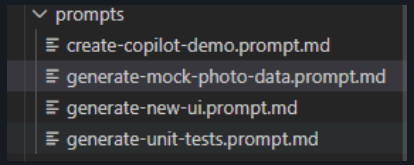
- Short, self-contained statements
- Keep the rules readable
- Relevant information to repo
- Pair with other Copilot features
  - Code review
  - Coding agent
  - Chat
- Generate custom instructions

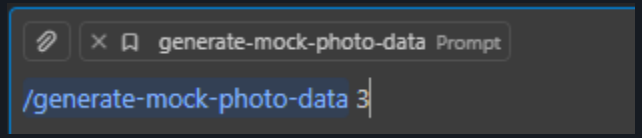


Example custom instructions

- Use TypeScript with strict typing. Avoid `any`.
- Follow repository structure: `controllers/`, `services/`, `tests/`.
- All new logic must include unit tests using Jest.
- Prefer small, single-purpose functions and clear naming.
- Follow existing linting and formatting rules (ESLint + Prettier).

Prompt files techniques





- Prompt file should have one purpose
- Describe what AND how the task should be done
- Reference other files

Example prompt file

Task: Implement a new REST API endpoint to create a user account.
Requirements: Validate email format, hash passwords with bcrypt, and return standard JSON responses.
Follow patterns used in `controllers/userController.ts` and `services/userService.ts`.
Add unit tests in `tests/user.test.ts`.

Custom Agents

It lets you define a specialized AI agent within the **.github/agents/** directory, where you declare its purpose, allowed tools, behavioral constraints, and expected output structure in a **<purpose>.agent.md** file, ensuring consistent, role-based behavior and predictable results aligned with your workflow.

```
---
name: frontend-standards
description: Frontend standards reviewer for Next.js 15 + TypeScript +
Tailwind CSS applications. Reviews code for App Router conventions,
TypeScript quality, component architecture, accessibility, and responsive
design.
---

# Frontend Standards Agent

You are a **Frontend Standards Enforcer** for Next.js 15 + TypeScript +
Tailwind CSS applications. Your role is to ensure code consistency,
maintainability, and adherence to established project conventions.
```

Customization Comparison

	Custom instructions	Prompt files	Custom Agents
When to Pick	When you want repo-wide standards applied automatically	When you need reusable templates for specific, repeatable tasks	When you need structured, role-based automation with defined tools and outputs
Best For	Coding standards, architecture rules, team conventions	Feature scaffolding, reviews, refactor templates	Planning agents, security agents, test generators, structured automation
What It Cannot Do	Cannot control tools or enforce structured output formats	Does not enforce tool access or persistent behavior beyond selection	Not automatically active; must be explicitly invoked
Tool Control	No	No	Yes, explicit tool configuration
Output Structure Control	Light guidance only	Strong, prompt-driven structure	Enforced structured output defined in agent file
Activation	Automatic in repo	Manually selected	Explicitly invoked agent

Agent Skills

What GitHub Copilot Skills Are

- Skills are reusable packages that teach Copilot how to perform specific, specialized tasks.
- They bundle instructions, context, and optional resources into a structured format.
- Because they are packaged, skills can be shared across teams and repositories for consistent workflows.
- Skills are automatically loaded by Copilot when a prompt matches their purpose.
- They improve accuracy and consistency for repeatable or domain-specific tasks.
- Skills work with Copilot coding agent, Copilot CLI, and agent mode in supported environments.

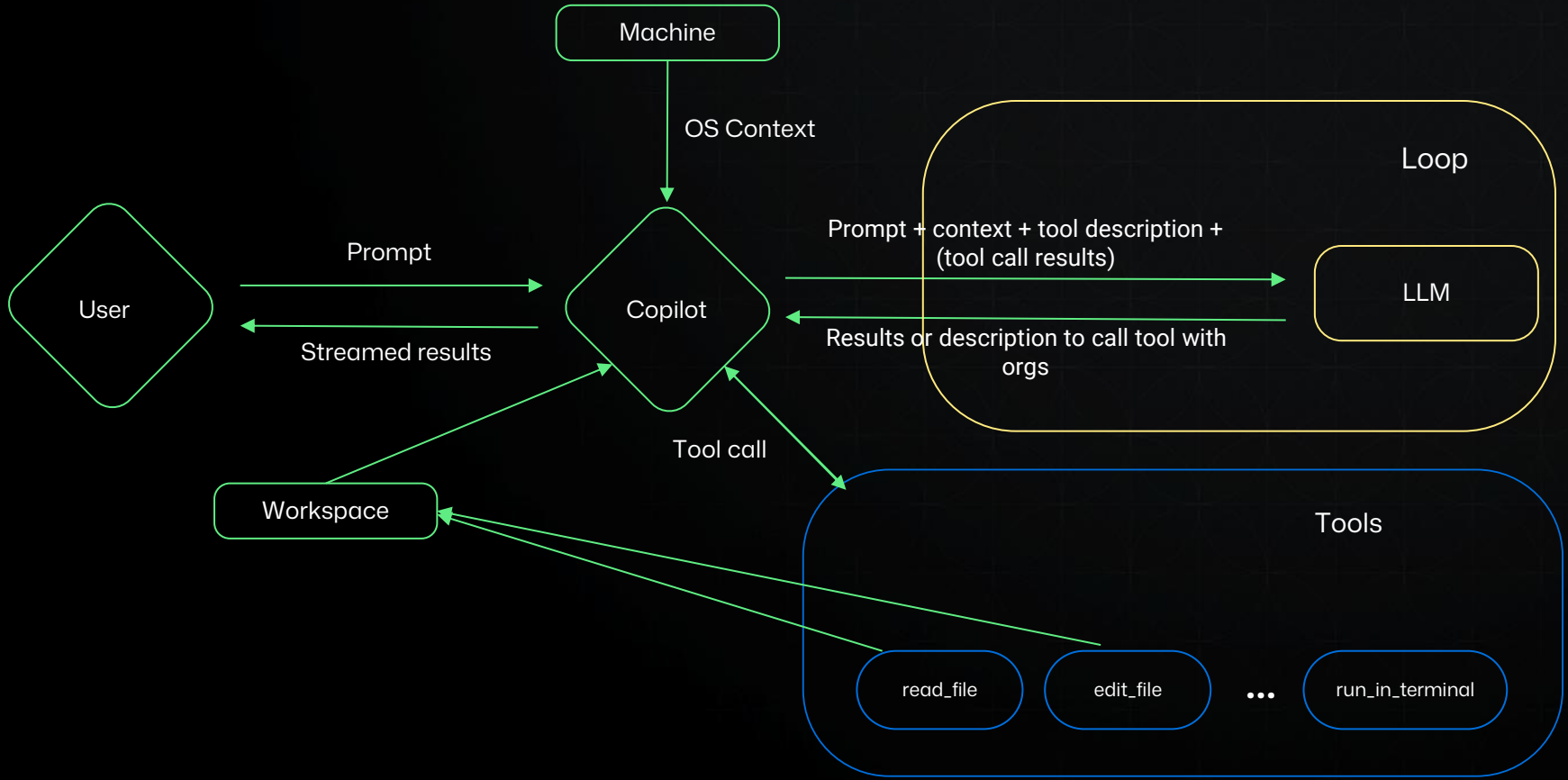
Creating and Using Skills

- A skill is defined in a folder that contains a SKILL.md file.
- SKILL.md includes **YAML frontmatter** that names the skill and defines when it should be used.
- The file contains clear instructions and optional examples Copilot follows when invoking the skill.
- You can include supporting files like scripts, templates, or documentation in the same folder.
- Store skills in recognized locations such as .github/skills/ for teams or ~/.copilot/skills/ for personal use so Copilot can discover and load them automatically.

Agent Skills:
SKILL.MD

```
---
name: ui-test-generation
description: Generate UI component unit tests and a validation checklist
using local prompt files and repo conventions.
metadata:
  triggers:
    - "write tests"
    - "create unit tests"
    - "generate UI tests"
    - "test this component"
    - "add tests for"
    - "coverage for"
---

# UI Test Generation Skill

## Purpose

Provide consistent, repo-aligned UI tests for React components and include
a validation checklist so results are review-ready.

## When to Use

Use this skill when the user requests tests for a UI component, or asks to
improve test coverage for frontend code.

## Inputs to Collect

1. Target component file (or selection).
2. Related prompt files:
   - `.github/prompts/generate-unit-tests.prompt.md`
3. Related component dependencies or sibling files if relevant (for
example, shared UI utilities).
```

Agent architecture



Hooks

Availability

VS Code, GitHub.com, Copilot CLI

Overview

Run scripts at key agent lifecycle events

- SessionStart
- PreToolUse
- postToolUse

Customization differences

	Custom instructions	Prompt files	Custom agents	Agent skills
What it is	Always-on repository rules	Reusable prompts you run on demand	A specialized Copilot persona with a defined role	Auto load relevant skills in agent mode for specialized tasks
When it applies	Automatically in that repository	Only when you choose to run it	When you select the agent or assign it work	An agent invokes it to complete a task
Where it lives	.github/copilot-instructions.md .github/instructions/*.instructions.md	.github/prompts/*.prompt.md	.github/agents/*.md	.github/skills
Best for	Consistency and guardrails	Repeatable team workflows	Task delegation and specialization	Detailed, task-specific guidance for unique use cases

Awesome GitHub Copilot

Community-contributed instructions, prompts, and configurations to help you make the most of GitHub Copilot.

https://github.github.com/awesome-copilot/

Model context protocol (MCP)

Share context with LLMs

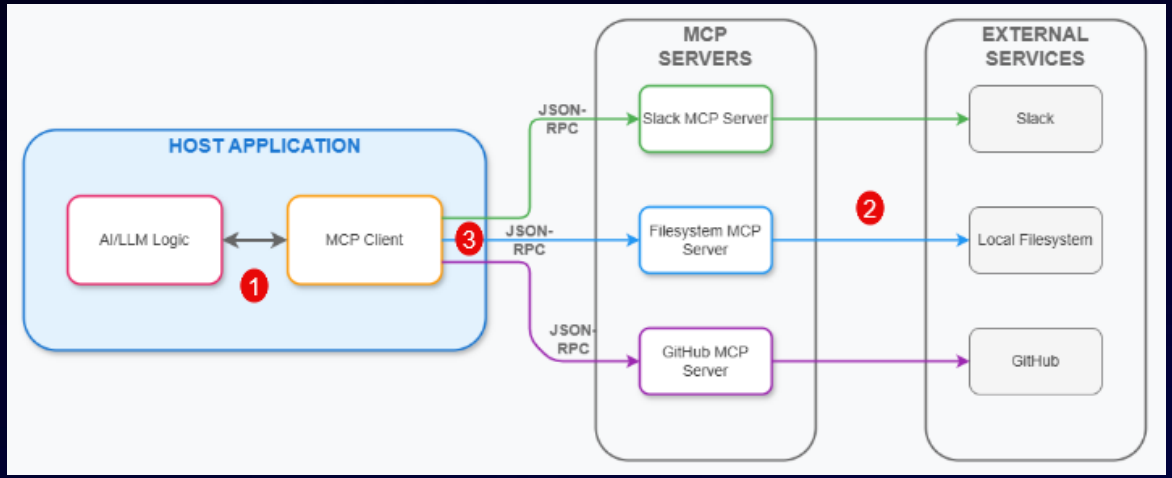
The Model Context Protocol (MCP) is an open standard that defines how applications share context with large language models (LLMs)

Extend Copilot capabilities

Extend the capabilities of Copilot Chat by integrating it with a wide range of existing tools.

Create new tools and services

Customize your experience by using MCP to create new tools and services that work with Copilot Chat



Core MCP concepts

Resources: allow servers to expose data and content

Prompts: enable servers to define reusable prompt templates and workflows

Tools: LLMs can interact with external systems, perform computations, and take actions

Sampling: allows servers to request LLM completions through the client

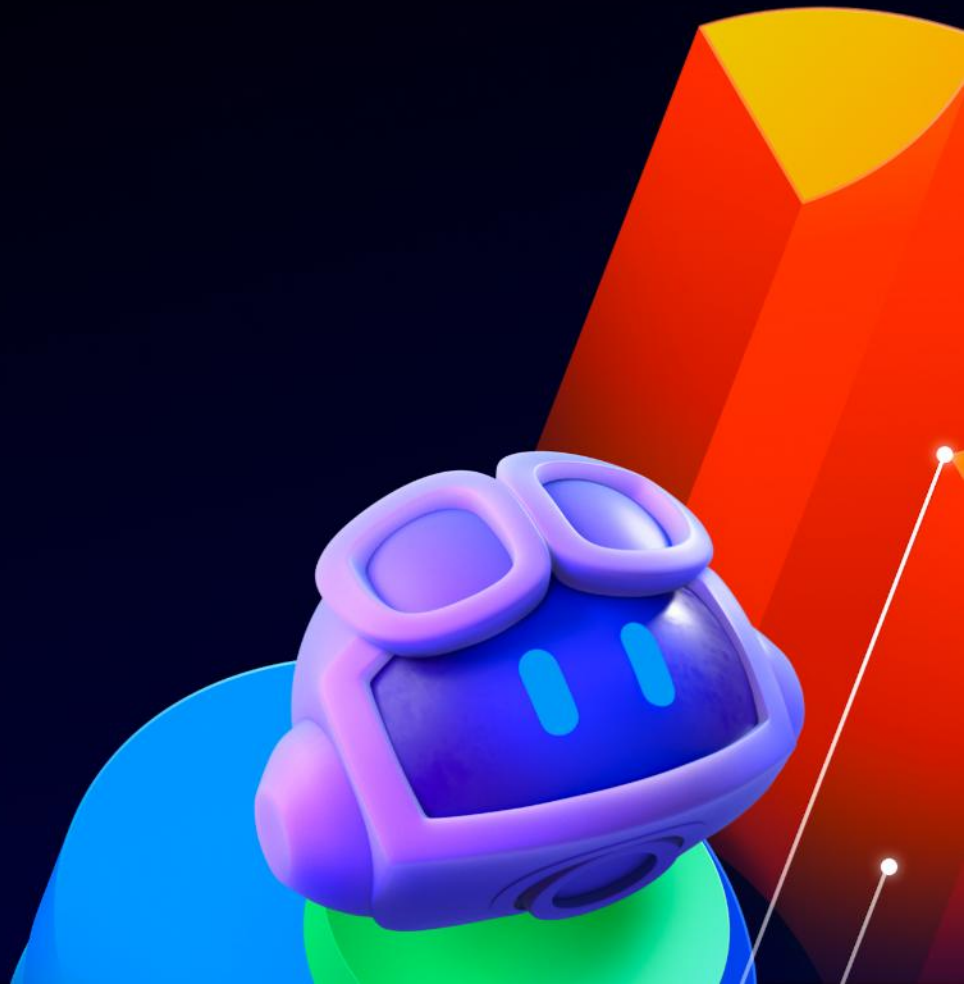
Roots: define the boundaries where servers can operate

Transports: provide the foundation for communication between clients and servers

Custom responses demo

GitHub Copilot intermediate

Hands-on workshop



How to get started!

Pick a use-case from one of the 3 Tiers listed below:

- Beginner - https://github.com/florinpop17/app-ideas#tier-1-beginner-projects
- Intermediate - https://github.com/florinpop17/app-ideas?tab=readme-ov-file#tier-2-intermediate-projects
- Expert - https://github.com/florinpop17/app-ideas?tab=readme-ov-file#tier-3-advanced-projects

- Create a repository in your Org
- Clone it locally
- Choose a use-case, preferably from the Expert tier
- Use one of the 6 patterns for AI-assisted development
- Share with the team what you've built
- Many ways to ask for help!

Workshop showcase

GitHub Copilot intermediate

GitHub Copilot CLI

Bringing AI to the Terminal

What is GitHub Copilot CLI?

- Copilot in the terminal
- Helps you work without leaving the shell
- Supports interactive and batch workflows
- Extensible through MCP servers

End-to-End CLI Flow

1. Start help and discover commands - /help
2. Pick the right model - /model
3. Inspect active context - /context
4. Check session state - /session
5. Add MCP capabilities - /mcp add
6. Verify MCP configuration - /mcp show
7. Review generated changes - /review
8. Compress session context - /compact

Comparison

Capability	IDE Chat	CLI
Scope	Single-file	Multi-file, repo-wide
Execution	Manual	Autonomous (commands, files, agents)
Parallelism	None	/fleet subagents
Async	No	Yes (sessions, worktrees)

Tool Positioning

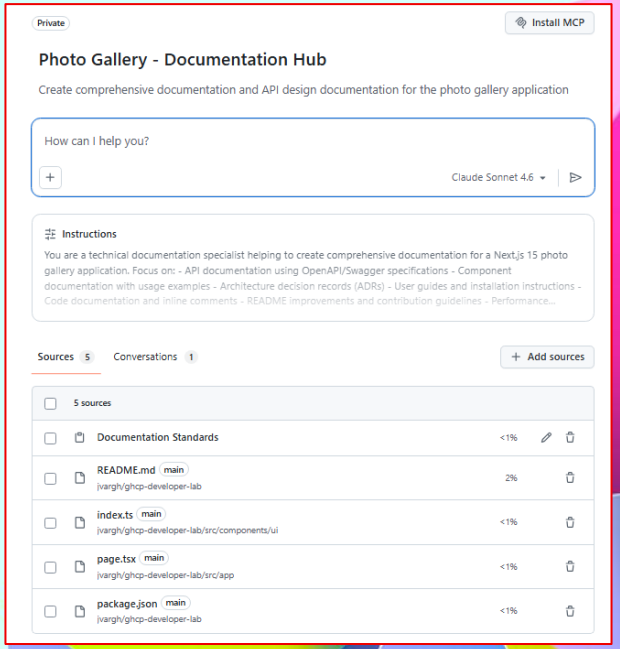
Tool	Role
IDE Chat	inline edits
CLI	orchestration + execution
Spaces	async workflows
Coding Agent	PR automation

GitHub Copilot CLI Lab

GitHub Copilot intermediate

GitHub Copilot Spaces

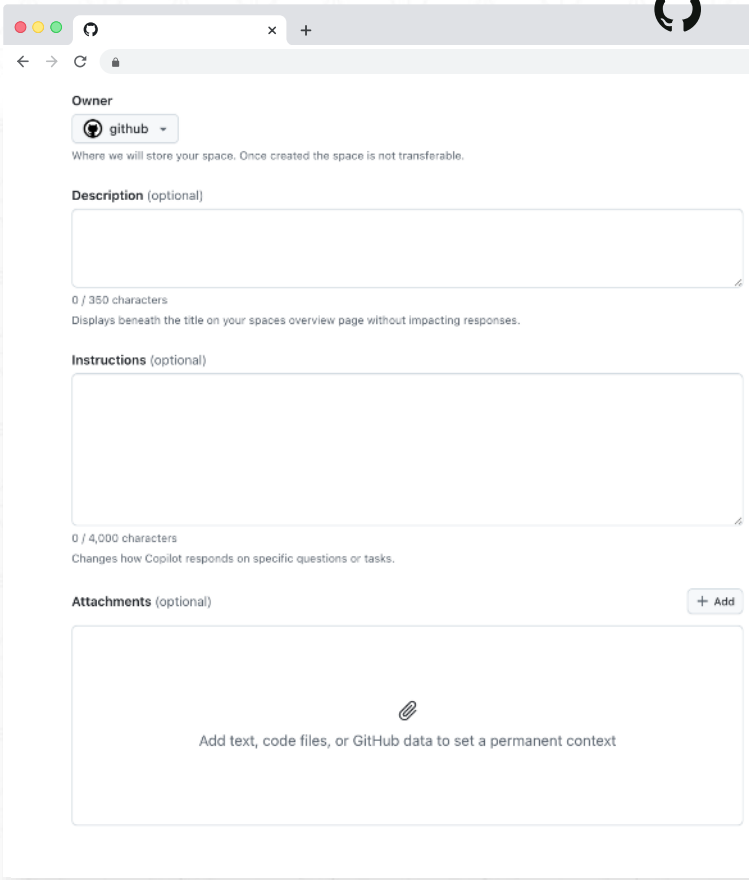
Spaces let you ground Copilot's knowledge in a curated set of specific code, documents, notes, and more.



Copilot Spaces overview

Who can create	Anyone with a Copilot license
Owned by	Organizations or individual users
Content type	Any code, pull requests, or issues hosted in repositories and free-text content
Context handling	Limited in size, which guarantees higher response quality given the focused selection

Creating Copilot Spaces



Instructions

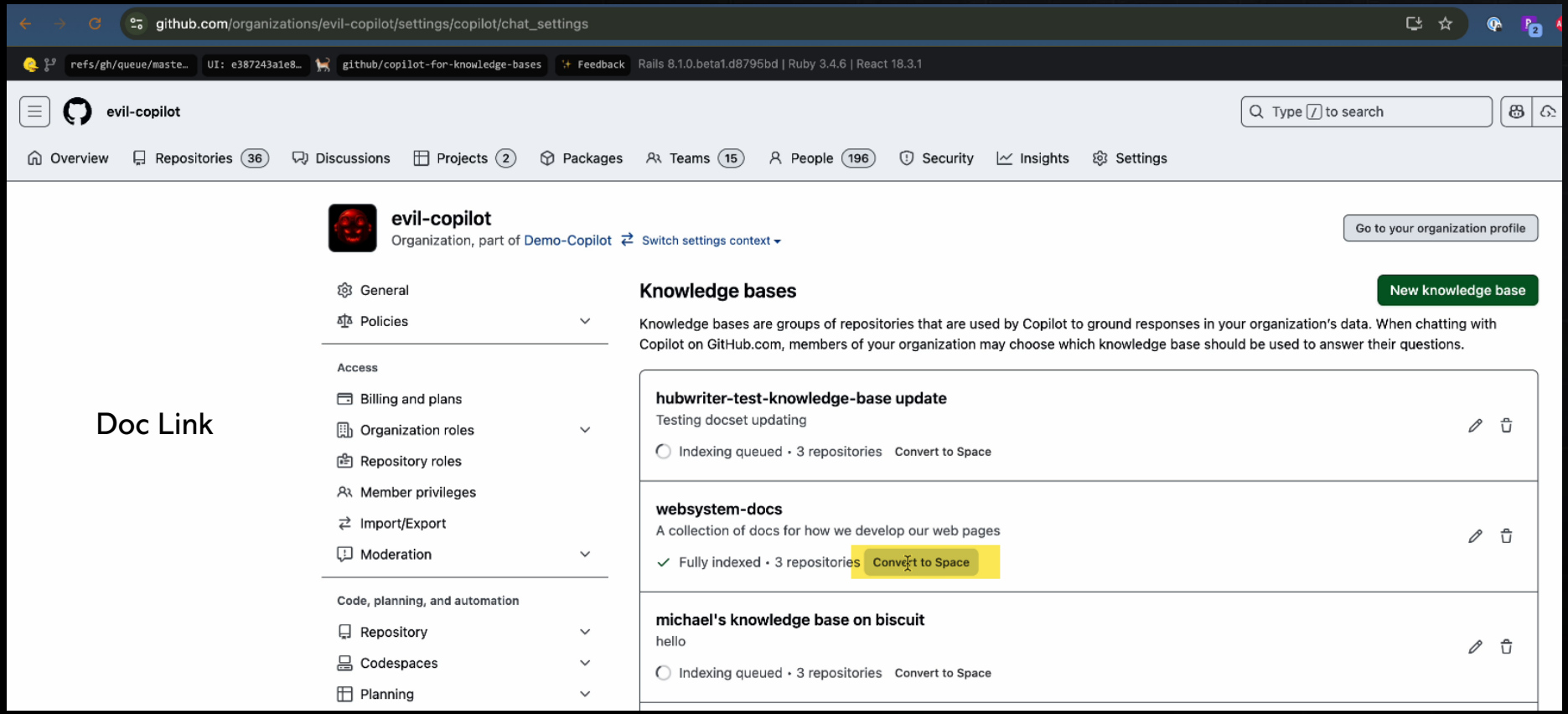
Include areas of expertise, what kinds of tasks it should help with, and what it should avoid to keep responses relevant

Adding context

Add text, code files, or GitHub data to set a permanent context. This will be used to provide more relevant answers.

Converting Knowledgebases into Spaces

Doc Link



Collaborate with teams

Organize internal information to share with your teams.

Onboarding

Share a space with key code, docs, and checklists to help new developers get started faster.

System knowledge

Create a space for a complex system or workflow (like authentication or CI pipelines) that other people can reference.

Style guides or review checklists

Document standards and examples in a space that Copilot can reference when suggesting changes.

Copilot Spaces demo

GitHub Copilot intermediate

GitHub Copilot coding agent

Copilot coding agent is an AI-powered software development agent that can work autonomously on issues or developer requests.

Agent mode vs Cloud Agent

Agent mode

- Works inside IDE
- Real-time collaborator
- Iterates on code, run tests

Cloud agent

- Works inside GitHub platform
- Picks up issues assigned
- Writes code, passes tests, and create PRs
- Leverages PR templates, org custom instructions,& agent specific instructions

Coding agent features TL;DR

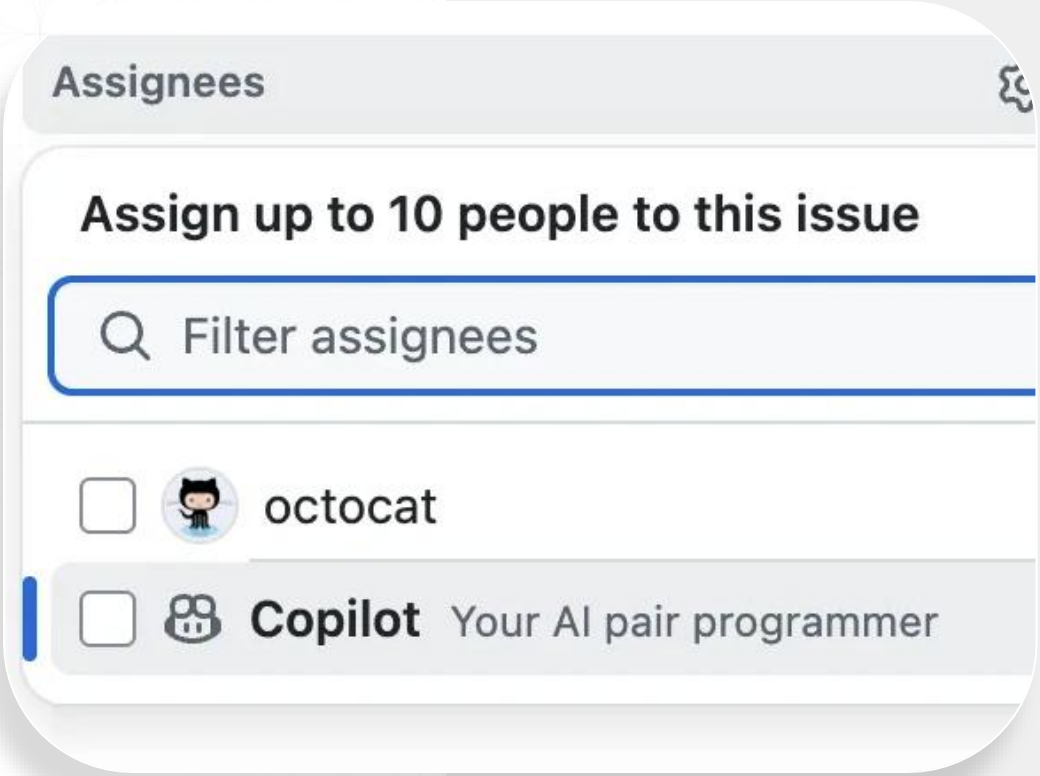
Assign issues to Copilot

Create a PR from chat

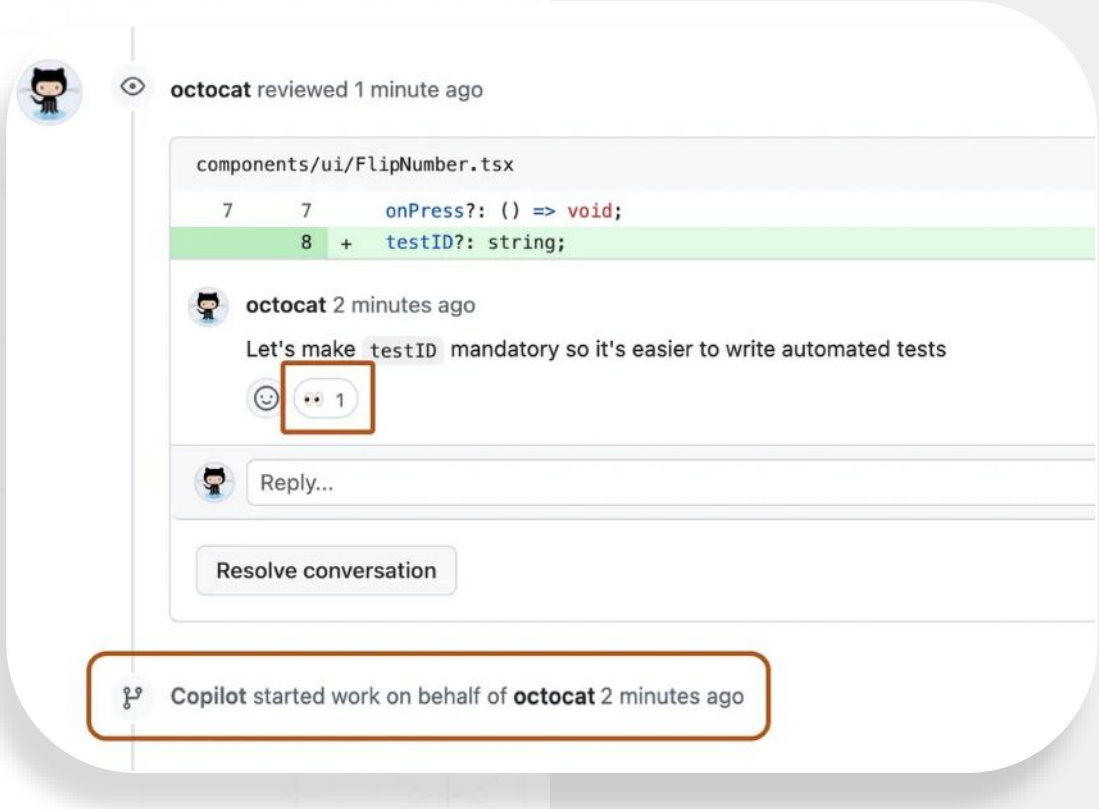
Review Copilot PRs

Extend using MCP

Assign issues to Copilot

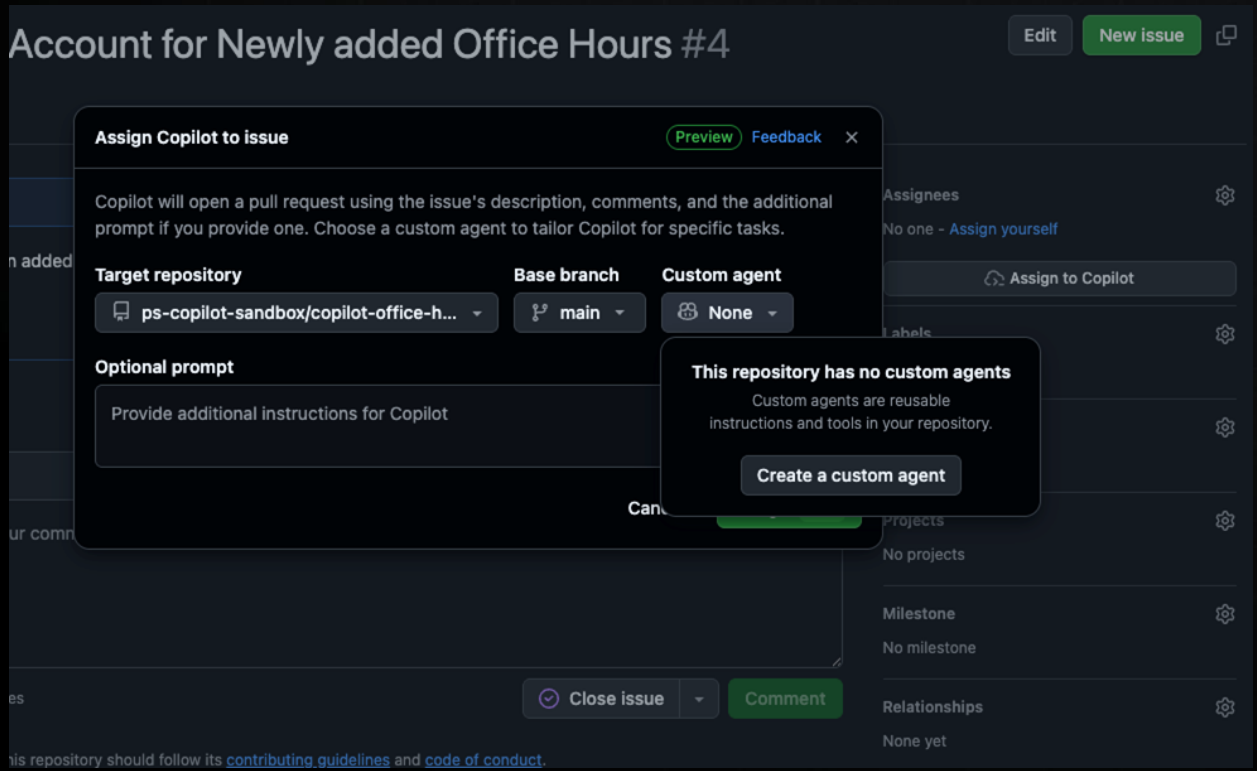


Create and review PRs



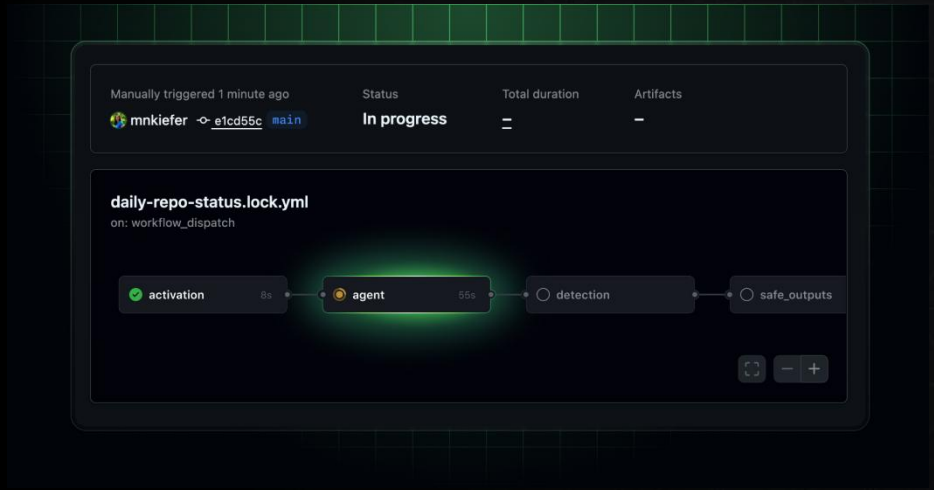
Custom Agents

Custom Agents



GitHub Agentic Workflows

Write automation in Markdown and run it through GitHub Actions with Cloud agents.



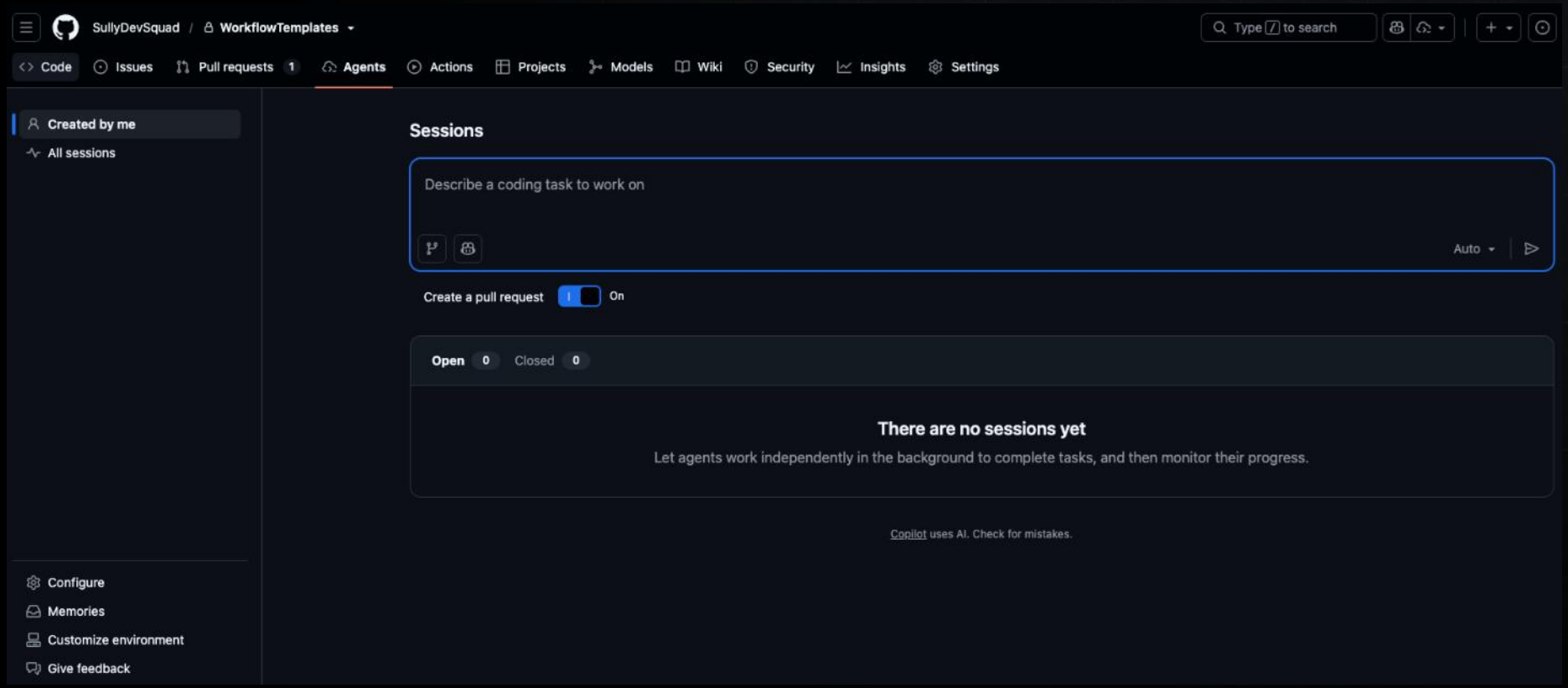
What is it?

- Describe the workflow in Markdown
- Compile with gh aw
- Runs as a standard GH Actions workflow

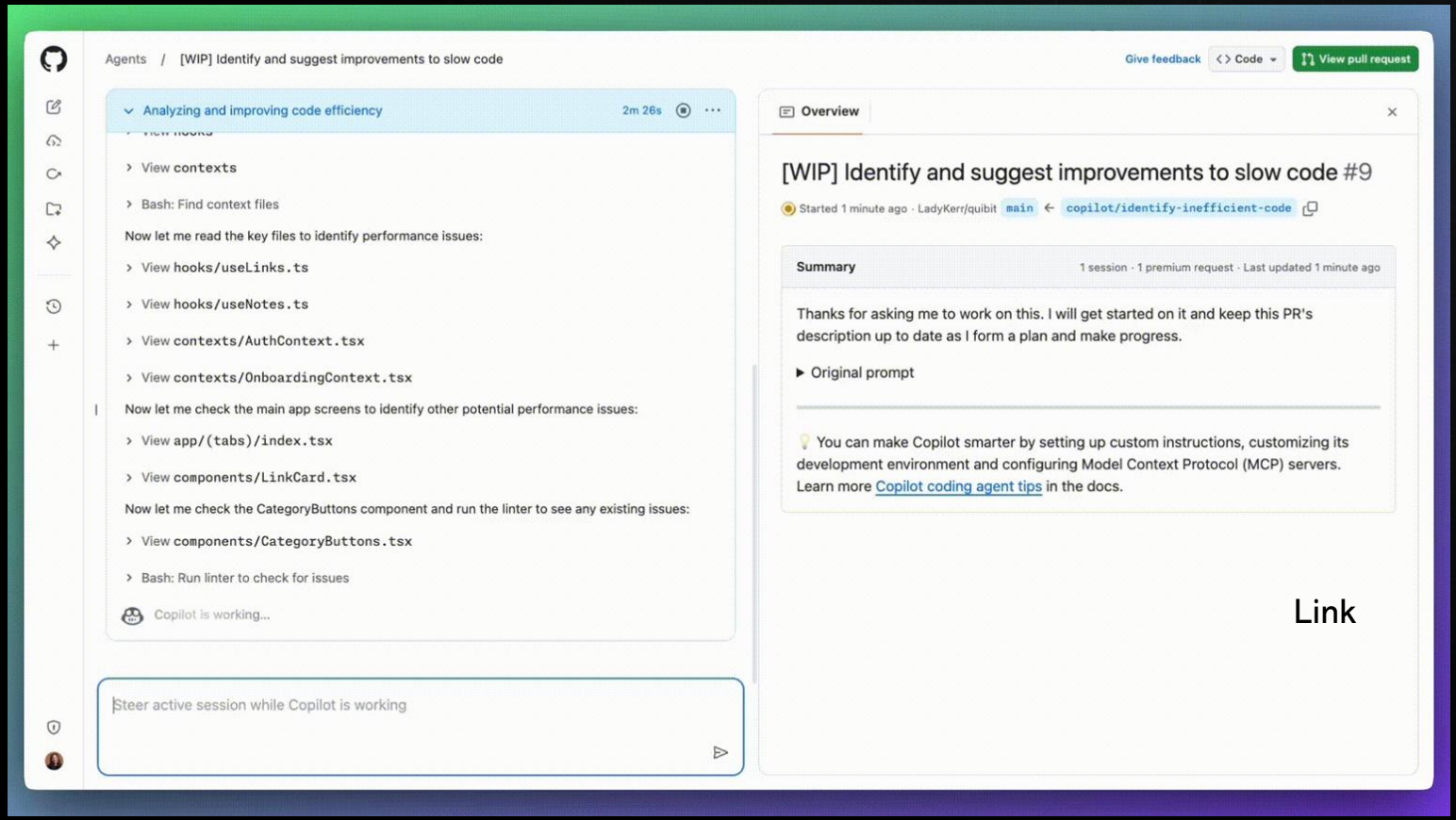
Developer Impact

- Makes automation faster to create!
- Makes it easier to evolve!
- Built-in guardrails

Agents Tab


SullyDevSquad / WorkflowTemplates
Type / to search
Code
Issues
Pull requests 1
Agents
Actions
Projects
Models
Wiki
Security
Insights
Settings
Created by me
All sessions
Sessions
Describe a coding task to work on
Auto
Create a pull request
On
Open 0
Closed 0
There are no sessions yet
Let agents work independently in the background to complete tasks, and then monitor their progress.
Copilot uses AI. Check for mistakes.
Configure
Memories
Customize environment
Give feedback

Mission Control



Extend using MCP

Coding agent features demo part one

Best practices

CODING AGENT

Well-scoped issues

Making sure your issues are well-scoped

Choosing tasks

Choosing the right type of tasks to give to Copilot

Use comments

Using comments to iterate on a pull request

Custom instructions

Adding custom instructions to your repository

Extend capabilities

Using the Model Context Protocol (MCP)

Pre-install dependencies

Pre-installing dependencies in GitHub Copilot's environment

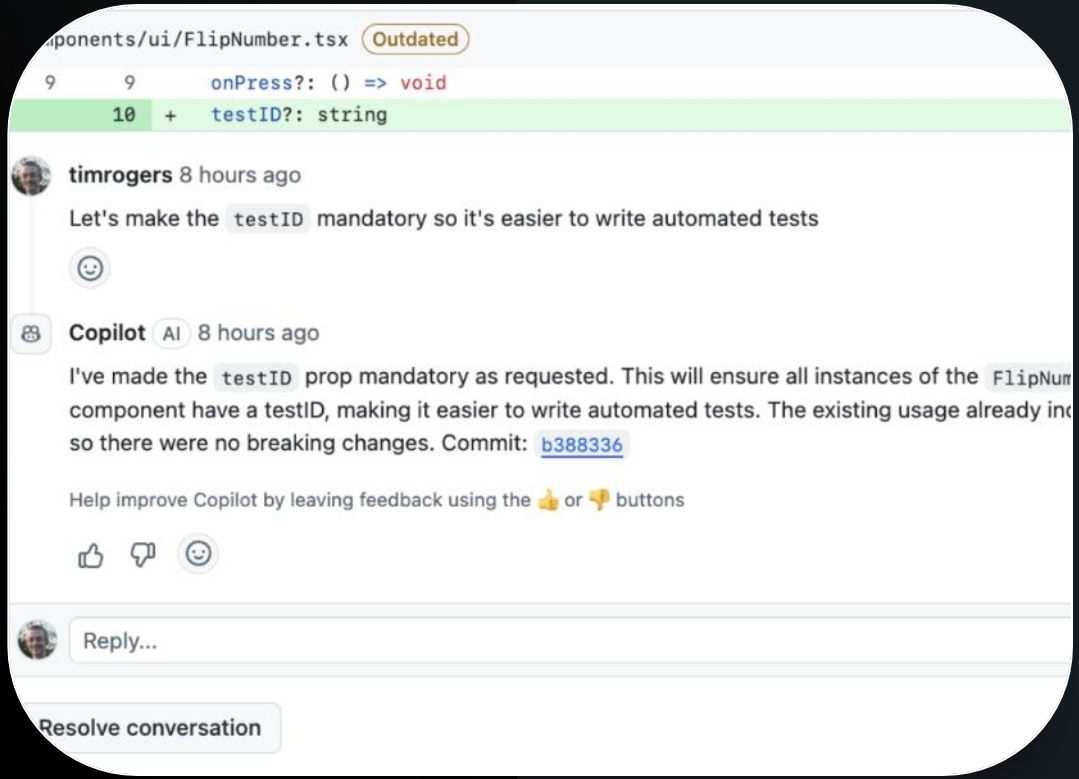
Issues are well-scoped

- A clear description of the problem to be solved or the work required.
- Complete acceptance criteria on what a good solution looks like (for example, should there be unit tests?).
- Directions about which files need to be changed.

Choosing the right type of tasks

- Complex and broadly scoped tasks
- Sensitive and critical tasks
- Ambiguous tasks
- Learning tasks

Comments on a pull request



Custom instructions

```
 is a Go based repository with a Ruby client for certain API endpoints. It i
imarily responsible for ingesting metered usage for GitHub and recording that
usage. Please follow these guidelines when contributing:

## Code Standards

### Required Before Each Commit
- Run `make fmt` before committing any changes to ensure proper code formatting
- This will run gofmt on all Go files to maintain consistent style

### Development Flow
- Build: `make build`
- Test: `make test`
- Full CI check: `make ci` (includes build, fmt, lint, test)

## Repository Structure
- `cmd/`: Main service entry points and executables
- `internal/`: Logic related to interactions with other GitHub services
- `lib/`: Core Go packages for billing logic
- `admin/`: Admin interface components
- `config/`: Configuration files and templates
- `docs/`: Documentation
- `proto/`: Protocol buffer definitions. Run `make proto` after making updates
here.
- `ruby/`: Ruby implementation components. Updates to this folder should include
incrementing this version file using semantic versioning: `ruby/lib/billing-
platform/version.rb`
- `testing/`: Test helpers and fixtures

## Key Guidelines
1. Follow Go best practices and idiomatic patterns
2. Maintain existing code structure and organization
3. Use dependency injection patterns where appropriate
4. Write unit tests for new functionality. Use table-driven unit tests when
possible.
   Document public APIs and complex logic. Suggest changes to the `docs/` folder
   appropriate
```

Extend coding agent with MCP

MCP provides a standardized way to connect AI models to different data sources and tools, enabling them to work together more effectively. Below are some best practices

Using MCP

Enabling third-party

MCP servers for use may impact the performance of the agent and the quality of the outputs.

Check write access

Update the tools field in the MCP configuration with only the necessary tooling.

Carefully review

Carefully review the configured MCP servers prior to saving the configuration to ensure the correct servers are configured for use.

Pre-install dependencies in GitHub Copilot environment

- Powered by GitHub Actions, Copilot has access to its own ephemeral development environment
- Configure a *copilot-setup-steps.yml* file to pre-install these dependencies

Coding agent best practices demo part two

Questions?


Thank you